Assignment 1: Email Analyzer

Problem Statement

A system receives user email addresses during registration.
You must write a program that analyzes an email string and extracts useful information.

Input

A single email string entered by the user.

Example:

```
john.doe2024@gmail.com
```

Tasks

1. Remove any leading or trailing spaces using `trim()`.
2. Check if the email contains the `@` symbol.
3. Extract the **username** (before `@`) using `substring()` and `indexOf()`.
4. Extract the **domain name** (after `@`).
5. Check if the email belongs to **gmail.com** using `equalsIgnoreCase()`.
6. Convert the entire email to **lowercase**.
7. Count the number of characters in the username using `length()`.
8. Check whether the username contains any numeric characters.
9. Replace all dots `.` in the username with underscores `_`.

Expected Output

```
Username: john.doe2024
Domain: gmail.com
Total characters in username: 12
Contains digits: Yes
Modified username: john_doe2024
```

Assignment 2: Product Code Parser

Problem Statement

An e-commerce system stores product codes in the following format:

```
CAT-ITEMCODE-YEAR
```

Example:

```
ELC-TV-2023
```

Tasks

1. Split the product code into parts using `split()`.
2. Extract:
   - Category
   - Product name
   - Manufacturing year
3. Convert category to uppercase.
4. Check whether the product name starts with `"TV"` using `startsWith()`.
5. Check if the code ends with `"2023"` using `endsWith()`.
6. Replace hyphens – with spaces.
7. Print the position of the first hyphen using `indexOf()`.

Expected Output

```
Category: ELC
Product: TV
Year: 2023
Starts with TV: Yes
Ends with 2023: Yes
Modified Code: ELC TV 2023
```

Assignment 3: Password Policy Validator

Problem Statement

Write a program that validates a password based on the following rules.

Password Rules

- Minimum length: **8 characters**
- Must contain **at least one uppercase letter**
- Must contain **at least one lowercase letter**
- Must contain **at least one digit**

Tasks

1. Check password length using `length()`.
2. Iterate through characters using `charAt()`.
3. Determine if the password contains uppercase letters.
4. Determine if the password contains lowercase letters.
5. Determine if the password contains digits.
6. Replace spaces in the password using `replace()`.

Output

Display whether the password is **valid or invalid**, and show the reason.

Example:

```
Password: Pass1234
Length valid: Yes
Contains uppercase: Yes
Contains lowercase: Yes
Contains digit: Yes

Password is VALID
```

Assignment 4: Sentence Formatter

Problem Statement

Users submit sentences with inconsistent formatting.

Example input:

```
java programming is FUN and powerful
```

Tasks

1. Remove leading and trailing spaces.
2. Convert the sentence to lowercase.
3. Capitalize the first letter of the sentence.
4. Replace `"fun"` with `"interesting"`.
5. Count the number of words using `split(" ")`.
6. Print the first and last word.

Expected Output

```
Formatted Sentence: Java programming is interesting and powerful
Total words: 6
First word: Java
Last word: powerful
```

Assignment 5: File Name Validator

Problem Statement

A system receives file names uploaded by users.

Example:

```
report_final_2024.pdf
```

Tasks

1. Check whether the filename ends with `.pdf`.
2. Extract the file name without extension.
3. Extract the extension using `lastIndexOf()` and `substring()`.
4. Replace underscores _ with spaces.
5. Convert the name to uppercase.
6. Check if the file name contains the word `"final"`.

Expected Output

```
File Name: report final 2024
Extension: pdf
Is PDF file: Yes
Contains 'final': Yes
```

Assignment 6: Word Frequency Counter

Problem Statement

Analyze a sentence and count how many times each word appears.

Example:

```
Java is powerful and Java is popular
```

Tasks

1. Convert the sentence to lowercase.
2. Split the sentence into words using `split(" ")`.
3. Compare words using `equals()`.
4. Print each word with its frequency.

Example Output

```
java : 2
is : 2
powerful : 1
and : 1
popular : 1
```